Elaborating Constructor Subtyping for Indexed Datatypes

Anonymous author
Anonymous affiliation

Abstract

(Dependent) pattern matching is a powerful feature of functional and dependently typed languages; however, by its own nature, it can generate proof obligations that are often trivial to discharge. This either leads to unsafe partial functions or, in safe settings, to tedious work. The problem is particularly acute when programs operate on subsets of an inductive datatype. Standard encodings pair data with propositions or refactor datatypes into indexed families, but both approaches complicate composition and proof automation.

Much of the difficulty arises from two facts: (1) the type system cannot trivially discard minor anti-unification obligations, (2) more acutely, the type system cannot express computational information about datatypes, and thus we cannot express these trivial simplifications as a type-level operation.

In this work, we present a subtyping discipline for constructor subsets; this substantially attacks the first problem. First-class signatures describe which constructors of a family are available. These signatures make the intended subset explicit at the type level, enabling pattern matching to rule out impossible branches without generating spurious obligations. Furthermore, we leverage this simple subtyping system as an elaboration mechanism in an extended version of Dagand and McBride's type theory.

Our results show that constructor subsets are expressive enough to capture common programming idioms, and that the extension to a dependently typed language is very simple. By extending Dagand & McBride's type theory, we show that constructor subsets can be expressed in pure computational settings, which enables reusing constructors in different type constructors, deciding about constructors, and even set-based operations on signatures.

We implement our simple encoding in Lambdapi and test it against a fragment of the type system. Later we elaborate and formalize the simple encoding into Dagand & McBride's extended type theory in Rocq.

2012 ACM Subject Classification Theory of computation → Program verification; Theory of computation → Type theory

Keywords and phrases Datatype, Type Constructors, Constructor Subtyping

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Subtyping is a fundamental concept across programming languages that enables the specialization of type definitions through hierarchical relationships between supertypes and subtypes. At a high level, a subtype describes a "smaller" collection of values than its supertype. While term-level subtyping is extensively implemented in many languages, subtyping between constructors of inductive types remains relatively unexplored, especially in the presence of dependent types.

A recurring difficulty arises when an inductive datatype is only partially available in a given context. For example, a function may be intended to consume lists that are known to be non-empty, or vectors whose length is positive. In most dependently typed languages the programmer faces a choice: either write an unsafe partial function, or thread propositions through the program and discharge proof obligations stating that certain constructors are



© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0

impossible. The latter approach leads to unreachable clauses in pattern matches and to proofs that are often routine but cumbersome.

One common technique is to maintain the original datatype together with a proposition that rules out some constructors. Another approach, illustrated in Figures 1 and 2, uses indexed datatypes to restrict which constructors may appear. Both techniques have well-known drawbacks. The first ties ordinary programs to logical predicates, complicating function composition and proof reuse. The second introduces additional indices and can interact badly with proof irrelevance and unification; it may also hinder the detection of unreachable branches when constructor information flows through computations.

Constructor subtyping has been explored in other settings. In O'Haskell [11], records and algebraic datatypes can be equipped with an optional subtyping relation inferred from the usage of constructors and fields. For example, a non-empty list type can be extended with an empty constructor to obtain a standard list type. More recent work on zero-cost constructor subtyping in Cedille [8, 7], on order-sorted inductive types and overloading [3, 2], and on algebraic subtyping for extensible records [9] shows how rich subtyping disciplines can be added to strongly typed systems. However, these systems are tailored to specific core calculi or to non-dependent settings, and do not directly address the combination of constructor subtyping with indexed families in a general dependently typed setting.

In the first part of the paper we focus on a simple but expressive fragment of constructor subtyping aimed at eliminating trivial proof obligations. Rather than allowing arbitrary constructor overloading, we work with *constructor subsets*: first-class types of the form $\{T :: \Phi\}$, where T is an inductive family and Φ is a finite list of its constructors. Intuitively, $\{T :: \Phi\}$ denotes those values of T whose *outermost (head) constructor*, when inspected in canonical form, is drawn from Φ. This is the information that coverage checking for pattern matching consumes directly, and it is exactly what makes types such as "non-empty lists" inhabited (the tail may still be empty). We restrict attention to subtyping relations induced by inclusion of constructor sets and by compatibility of the indices of T.

Our contributions are as follows.

- We define a subtyping discipline for constructor subsets as a conservative extension of a $\lambda\Pi$-style dependent type theory with inductive families [4]. The system includes typing and subtyping rules for constructor signatures and a dedicated elimination rule for pattern matching on signature types.
- We show, through a series of examples, that constructor subsets capture common programming idioms such as non-empty lists and simple invariants on inductive families, while avoiding the proof obligations that arise when these invariants are expressed with explicit propositions.
- We describe a two-layer prototype: a Lambdapi backend encoding of constructor signatures and a Haskell transpiler that translates a lightweight surface fragment into signature-aware LambdaPi terms. This architecture keeps programs concise while preserving a direct correspondence with Rules 1–7.

Structure of the paper.

Section 2 introduces the core type system and subtyping rules. Section 3 presents case studies illustrating how constructor subsets simplify programming with indexed families. Section 4 sketches the implementation and discusses practical considerations, and Section B concludes with directions for future work.

Many type systems are inspired by constructor subtyping to address constructor overloading, allowing the same constructor name to be shared by different datatypes, even in

```
Inductive list (A : Set) : Set :=
    | cons : A → list A → list A
    | empty : list A.

Definition head {A} (x : list A) :
    x <> empty A → A :=
    match x return x <> empty A → A with
        | cons _ h l ⇒ fun _ ⇒ h
        | empty _ ⇒
            fun f ⇒
                match (f eq_refl) with end
end.
```

Figure 1 Constructor exclusion in Coq with a proof obligation.

```
Inductive list' (A : Set) : bool → Set :=
    |cons' : forall x, A →
        list' A x → list' A true
    | empty' : list' A false.

Definition head' {A} (x : list' A true) : A :=
    match x with
        | cons' _ _ h l ⇒ h
    end.
```

Figure 2 Constructor exclusion in Coq using an indexed datatype.

pattern matching. However, its naive use can lead to problems with subject reduction, as observed by Frade [6]. Subject reduction is the property that during the normalization of a term its type is preserved; overloading constructors without care may violate this property. For the sake of simplicity, in this work we do not consider overloading between distinct datatypes. Instead we restrict subtyping to relations induced by subsets of constructors of a single inductive family. We believe this fragment is already expressive enough to avoid the trivial proof obligations discussed above, while keeping the meta-theory and implementation comparatively simple.

2 Type Rules

In this section, we introduce a simplified type system, enhanced with first-class signature subtyping support. This exposition assumes a basic familiarity with the concepts of type theory, especially dependent type theory. We use a standard typing judgment $\Gamma \vdash t : A$ for terms and a subtyping judgment $\Gamma \vdash A \sqsubseteq B$ between types. Contexts Γ map variables to types, and $Type$ denotes the universe of small types.

The subtyping relation is inspired by the subsumption rule of Aspinall and Compagnoni [1]:

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash A \sqsubseteq B}{\Gamma \vdash t : B}$$

We now fix some notation for telescopes and constructor environments. A *telescope* Δ is a sequence of typed variables

$$\Delta = (a_1 : A_1) \dots (a_n : A_n).$$

Given such a telescope and a type family T, we write $T\,\Delta$ for the application $T\,a_1\,\dots\,a_n$. When a telescope is used only to record the indices of a datatype we write Δ^*; in examples we use $Vector\,A\,n$ with $\Delta = (A : Type)(n : Nat)$ and $\Delta^* = (A, n)$ [12].

To refer to individual arguments in a telescope we use subscripts: if $\Delta = (a_1 : A_1) \dots (a_n : A_n)$ then Δ_i denotes $(a_i : A_i)$ and Δ_i^* denotes the corresponding index, for $1 \le i \le n$. For a term T we use the side condition $T_{\beta\eta}$ to indicate that we inspect the $\beta\eta$-normal form of T.

We write $\mathcal{C}_{\text{all}}$ for the finite set of static constructor declarations provided by the programmer. A static definition is a name of the constructor, notice that we do not care how constructors are labeled, we only need to know when they are equal, one may use unique names or hashes for that. The order of static definitions is stored in $\mathcal{C}_{\text{all}}$; in particular, mutually recursive definitions can be handled by first registering every constructor in $\mathcal{C}_{\text{all}}$ and only then checking the rules below. A *signature* Φ is a finite list of constructor names drawn from $\mathcal{C}_{\text{all}}$, and a type of the form $\{T\,\Delta^* :: \Phi\}$ maps each constructor in Φ to the restricted type $T\,\Delta^*$. We use a side predicate $\mathsf{AGAINST}(\Delta^*, \Delta'^*)$, defined later in this section, to ensure that indices in different constructors remain compatible. Because $\mathcal{C}_{\text{all}}$ is finite, the side conditions that test membership in $\mathcal{C}_{\text{all}}$ are decidable.

$$\textsc{Rule 1}\ \frac{\begin{array}{c}\Gamma \vdash T\,\Delta^* : Type \quad \Phi = (C_1, \dots, C_n) \quad (C_1 \dots C_n) \subseteq \mathcal{C}_{\text{all}} \\ \forall i,\ 1 \le i \le n,\ \Gamma \vdash C_i : \Delta_i \to T\,\Delta_i'^* \\ \forall i,\ 1 \le i \le n,\ \forall j,\ 1 \le j \le |\Delta^*|,\ \mathsf{AGAINST}(\Delta_j^*,\ \Delta_{i,j}'^*)\end{array}}{\Gamma \vdash \{T\Delta^* :: \Phi\} : Type}$$

$$\textsc{Rule 2}\ \frac{\Gamma \vdash T\,\Delta^* : Type}{\Gamma \vdash \{T\,\Delta^* :: \Phi\} \sqsubseteq T\,\Delta^*}$$

$$\textsc{Rule 3}\ \frac{\begin{array}{c}\Gamma \vdash T\,\Delta^* : Type \quad \Phi = (C_1, \dots, C_n) \qquad \Gamma \vdash C : \Delta_C \to T\,\Delta_C'^* \quad C \in \Phi \\ \Gamma \vdash \overrightarrow{u} : \Delta_C \qquad (T\,\Delta_C'^*[\overrightarrow{u}/\Delta_C])_{\beta\eta} = (T\,\Delta^*)_{\beta\eta}\end{array}}{\Gamma \vdash C\,\overrightarrow{u} : \{T\,\Delta^* :: \Phi\}}\ C \in \mathcal{C}_{\text{ALL}}$$

$$\textsc{Rule 4}\ \frac{(T\,\Delta^*) =_{\beta\eta} (T'\,\Delta^*) \quad \Phi' \subseteq \Phi}{\Gamma \vdash \{T'\Delta^* :: \Phi'\} \sqsubseteq \{T\Delta^* :: \Phi\}}$$

$$\textsc{Rule 6}\ \frac{\Gamma \vdash A' \sqsubseteq A \qquad \Gamma, x : A' \vdash B \sqsubseteq B'}{\Gamma \vdash (x : A) \to B \sqsubseteq (x : A') \to B'}$$

Rule 5 (algorithmic only).

We keep the following rule as an implementation-level checking convenience (not part of the declarative calculus above). It is derivable from standard application typing plus subsumption:

$$\textsc{Rule 5}\ \frac{\Gamma \vdash F : \Delta \to A \qquad \Gamma \vdash \overrightarrow{u} : \Delta \qquad \Gamma \vdash A[\overrightarrow{u}/\Delta] \sqsubseteq \{T\,\Delta^* :: \Phi\}}{\Gamma \vdash F\,\overrightarrow{u} : \{T\,\Delta^* :: \Phi\}}\ F \notin \mathcal{C}_{\text{ALL}}$$

A term annotated as $T_{\beta\eta}$ is considered in $\beta\eta$-normal form. In practice we only normalize enough to inspect the outermost constructor when applying Rule 3 (and the algorithmic Rule 5 check in the implementation); there is no global requirement that all terms be strongly normalizing. The choice of how aggressively to normalize is left to the implementation and presents the usual trade-off between completeness of simplification and performance.

It is trivial to see that cons Δ has type $\{\text{Vector } \Delta^* :: |\text{cons}\}$. One may either infer this type and then check it against some expected type T, or attempt to check cons Δ directly against T. In our prototype we follow the former strategy—first infer, then check—because it tends to reduce the number of normalisation steps needed during type checking.

Operational reading.

Under the usual canonical forms lemma for inductive values, a closed value of a signature type $\{T\ \Delta^* :: \Phi\}$ reduces (in weak head normal form) to a constructor application $C\ \overrightarrow{u}$ with $C \in \Phi$ and $(T\ \Delta'^*_C[\overrightarrow{u}/\Delta_C])_{\beta\eta} = (T\ \Delta^*)_{\beta\eta}$. Thus, signature types refine *which head constructors* can appear at a given program point, exactly the information required to remove unreachable branches in pattern matching.

The AGAINST rule ensures that indices can be properly generalized to avoid false positive subtyping that would result in empty types. Consider, for instance, a type signature T defined as:

$$T := \{\text{Vector } A\ 0\ ::\ |\text{cons}\}$$

Such a type T leads to trivial proofs in dependent pattern matching unification—precisely what we aim to avoid, since T has no inhabitants, as can be trivially observed. More generally, any type constructor B can be represented as the bottom type $\bot$ if it takes the form $\{B\ ::\ |\emptyset\}$.

The intuitionistic logical explosion can be obtained through these types, commonly represented as $\Gamma, B \vdash A$, where A is any proposition [10]. It is important to note that while $\{B\ ::\ |\emptyset\}$ is intuitionistically explosive, its subtype B remains inoffensive. This property offers significant potential for representing proofs while preserving consistency, even when dealing with bottom types. We will discuss these properties in greater detail in the following sections.

Now we can introduce the AGAINST rules that try to generalize indexed datatypes between constructors. Operationally, $\mathsf{AGAINST}(p, t)$ can be read as a *first-order matching* or *generalization* check: the signature indices p (which may contain variables) must be general enough to accommodate the constructor result indices t. This is a predicate, so indexed values of different constructors have to respect these rules. The equality $=_\alpha$ simply means the alpha-equivalence relation.

$$\frac{}{\mathsf{AGAINST}(\emptyset, \emptyset)} \qquad \frac{\mathsf{AGAINST}(\Delta, \Delta') \quad v\ is\ Var}{\mathsf{AGAINST}(v\ ...\ \Delta, c\ ...\ \Delta')}$$

$$\frac{\mathsf{AGAINST}(\Delta, \Delta') \quad (c, c') \subseteq \mathcal{C}_{\text{all}} \qquad c\ \Delta^c =_\alpha\ c'\Delta^{c'}}{\mathsf{AGAINST}((c\ \Delta^c)\ ...\ \Delta, (c'\ \Delta^{c'})\ ...\ \Delta')}$$

Rule 2 allows us to forget signature information at the same index instance: any value of type $\{T\ \Delta^* :: \Phi'\}$ can be viewed as a value of the underlying type $T\ \Delta^*$. In this sense

constructor subsets behave like *phantom types* [5]: the extra information carried by the signature is present at type-checking time but is not reflected in the runtime representation of values.

Though phantom types are not widely known across programming languages—being primarily used in Haskell—they possess important properties for representing types that carry no information during runtime; that is, they are purely erased types [5]. In our system there is no dedicated elimination rule for pure phantom signatures, and Rule 7 only permits case analysis on signatures that still expose constructors.

A constructor C must adhere to certain restrictions in its definition. For example, the signature (i.e., a type $\{T\ \Delta^*\ ::\ \Phi\}$) cannot occur freely in C. This restriction prohibits constructor types such as $\text{succ} : \{\text{nat} :: |\text{succ} \mid 0\} \to \{\text{nat} :: |\text{succ} \mid 0\}$, which would lead to problematic self-reference, as succ would require itself in its own definition.

It is important to note that even $succ : nat \to nat$ constitutes an invalid definition in this system. This is because the predecessor is a phantom type; therefore, for C to satisfy the inductive criteria, a signature must occur freely only in the positive position in the Δ of succ.

Remark (inductive well-formedness).

The previous paragraph sketches the kind of restrictions that an implementation must enforce so that inductive definitions remain well-founded (e.g. positivity/strictness conditions and well-scoped use of signatures in recursive arguments). Our prototype enforces a conservative check in the spirit of standard positivity conditions; however, we do not attempt to present a complete account of inductive well-formedness for signatures in this paper.

We do not cover index matching (from dependent (co)pattern matching) rules and conversion details in this work, as they are beyond our current focus. We leave the implementation of these principles to the authors' discretion.

$$\textsc{Rule 7}\ \frac{\begin{array}{c}\Gamma \vdash T\ \Delta^* : Type \quad \Gamma \vdash Q : Type \qquad \Phi = (C_1, ..., C_n) \qquad \Gamma \vdash M : \{T\ \Delta^* :: \Phi\} \\ \forall i,\ 1 \le i \le n,\ \Gamma \vdash C_i : \Delta_i \to T\ \Delta_i'^* \qquad \forall i,\ 1 \le i \le n,\ \Gamma \vdash N_i : \Delta_i \to Q\end{array}}{\Gamma \vdash case\ M\ of\ Q\ \{C_i\ \Delta_i => N_i\ \Delta_i, ...\} : Q}$$

It's easy to see that the first-class $\{T\Delta :: \Phi\}$ corresponds to the types discussed above with type constructors specialization. The advantage is that eliminates any form of proposition holding these types. In proof assistants like Coq, this could be seen as the elimination of some proof obligation steps.

▶ **Theorem 1** (Head-constructor refinement). *Let T be an inductive family with constructor set C, and let $S \subseteq C$. Define $D : T \to Type$ by inspecting only the* head constructor *of the canonical form: for each constructor application $C'\ \overrightarrow{u}$,*

$$D(C'\ \overrightarrow{u}) \equiv \begin{cases} \top & \text{if } C' \in S, \\ \bot & \text{if } C' \notin S. \end{cases}$$

Then there is a definable map

$$f : \{T :: S\} \to \Sigma(x : T), D(x).$$

Moreover, in any host theory that validates refinement introduction (from $x : T$ and $D(x)$ to $x : \{T :: S\}$), this map is part of an isomorphism $\{T :: S\} \simeq \Sigma(x : T), D(x)$.

Proof sketch. Define $f(x) \coloneqq (x, \star)$. To justify $\star : D(x)$, use the canonical-forms property for signature types: if $x : \{T :: S\}$ is a value, its canonical form is $C\ \overrightarrow{u}$ with $C \in S$, hence $D(C\ \overrightarrow{u}) \equiv \top$.

For the converse direction, assume refinement introduction: from $(y, d) : \Sigma(x : T), D(x)$ we obtain $y : \{T :: S\}$ and define $g(y, d) \coloneqq y$. Then f and g are mutually inverse. ◀

$$\xi\text{-}\cdot\text{APP}_1\ \frac{L \longrightarrow L'}{L \cdot M \longrightarrow L' \cdot M} \qquad \xi\text{-}\cdot\text{APP}_2\ \frac{M \longrightarrow M'}{V \cdot M \longrightarrow V \cdot M'}$$

$$\xi\text{-}\beta\ \frac{}{(\lambda x \Rightarrow N) \cdot V \longrightarrow N[x := V]}$$

$$\xi\text{-}case\ \frac{v \longrightarrow v'}{case\ v\ of\ Q\ \{C_i\ \Delta_i \Rightarrow N_i\ \Delta_i,...\} \longrightarrow case\ v'\ of\ Q\ \{C_i\ \Delta_i \Rightarrow N_i\ \Delta_i,...\}}$$

$$\xi\text{-}case'\ \frac{}{case\ (C_i\ \Delta')\ of\ Q\ \{C_i\ \Delta_i \Rightarrow N_i\ \Delta_i,...\} \longrightarrow N_i\ \Delta'}$$

Metavariables M, N, S range over terms; V, W range over values (constructors in WHNF). In the rule ξ-case above, the symbols v, v' denote arbitrary terms (not values).

▶ **Theorem 2** (Progress). *If $\cdot \vdash M : \{T\ \Delta^* :: \Phi\}$, then either M is a value or there exists M' such that $M \longrightarrow M'$.*

Proof sketch. We reason by structural induction on M and case analysis on its form.

- *Application* $M = M_1 \cdot M_2$. If either M_1 or M_2 is not a value, the induction hypothesis yields M_i' with $M_i \longrightarrow M_i'$, and we use the corresponding congruence rule ξ-app$_i$ to obtain a step for M. Otherwise both M_1 and M_2 are values. If M_1 is a lambda abstraction $\lambda x.N$, we can perform a β-reduction step. If M_1 is a constructor, then M is already a value (an applied constructor) and the conclusion holds.
- *Case expression* $M = case\ v\ of\ Q\ \{C_i\ \Delta_i \Rightarrow N_i\ \Delta_i, \ldots\}$. If v is not a value, the induction hypothesis gives v' with $v \longrightarrow v'$, and we use rule ξ-case. If v is a value, then by typing rule 7 we have $v : \{T\ \Delta^* :: \Phi\}$. There must be some pattern $C_i : \Delta_i$ that matches v: by Rule 3 we know that the head constructor of v lies in Φ, and by Rule 7 we have a corresponding branch for every constructor listed in Φ. In this situation we can apply rule ξ-case' and M takes a step.
- *Other forms.* For variables, constructors, and abstractions the canonical forms analysis from the typing rules shows that well-typed closed terms of signature type are either values or reduce by one of the cases above.

In all cases, a closed term of type $\{T\ \Delta^* :: \Phi\}$ is either a value or can take a reduction step. ◀

▶ **Theorem 3** (Preservation). *If $\Gamma \vdash M : R$ and $M \longrightarrow M'$, then $\Gamma \vdash M' : R$.*

Proof sketch. We proceed by induction on the evaluation derivation, considering the last reduction rule used.

- *β-reduction.* If $M = (\lambda x.N) \cdot V$ and $M' = N[x := V]$, typing gives $\Gamma \vdash \lambda x.N : (x : A) \to B$ and $\Gamma \vdash V : A$. By the substitution lemma we obtain $\Gamma \vdash N[x := V] : B[x := V]$, which is the same type as M.

- *ξ-app$_1$ and ξ-app$_2$.* In these cases we reduce a proper subterm of M. The induction hypothesis states that the reduced subterm preserves its type; reapplying the application typing rule reconstructs a typing derivation for M' with the same type R.
- *ξ-case.* Here $M = \text{case } v \text{ of } Q\ \{\dots\}$ and $M' = \text{case } v' \text{ of } Q\ \{\dots\}$ with $v \longrightarrow v'$. By the induction hypothesis $\Gamma \vdash v' : \{T\ \Delta^* :: \Phi\}$, and Rule 7 then re-establishes $\Gamma \vdash M' : Q = R$.
- *ξ-case'.* In this case $M = \text{case } (C_i\ \Delta') \text{ of } Q\ \{\dots\}$ reduces to $M' = N_i\ \Delta'$. Typing rule 7 requires that for each branch we have $\Gamma \vdash N_i : \Delta_i \to Q$, and that $C_i : \Delta_i \to T\ \Delta^*$. Instantiating with the actual arguments Δ' shows that M' has type Q, the same type as M.

All other reduction rules follow the same pattern: either a proper subterm reduces and the induction hypothesis applies, or a local computation such as β-reduction is justified by the typing rules. In each case the result has the same type R as the original term. ◀

3 Case Examples

We introduce a minimal language and its implementation to demonstrate the subtyping system. The abstract syntax of terms is sketched in Figure 3; in the examples below we use a lightweight ML-style surface syntax.

$$\begin{aligned}\text{Term} ::= {} & \text{Var}(x) \mid \text{App}(f, a) \mid \text{Lam}(x, e) \mid \text{Pi}(x, A, B) \\ & \mid \text{Constr}(T, \overrightarrow{c}) \mid \text{Match}(e, T, \overrightarrow{p \to e'}) \mid \text{Notation}(e, T)\end{aligned}$$

Figure 3 Term syntax of the example language

3.0.0.1 Mini-language overview.

Programs consist of a sequence of *static declarations* followed by regular statements. Static declarations register constructors in $\mathcal{C}_{\text{all}}$ (Rule 1), whereas dynamic statements bind names to terms typed by the rules from Section 2. Terms use the usual λ-abstractions, function types, applications, and pattern matches; the only novelty is that constructor signatures $\{T :: \Phi\}$ appear explicitly in types. These informal descriptions suffice for the examples below; the full concrete syntax is implemented in the prototype but omitted here.

We exploit the capability to operate with dynamically defined constructor subsets without requiring trivial proof obligations. Since datatype signatures are first-class inhabitants in our system, we can instantiate varying constructor sets through definition aliasing. The following declarations register constructors for lists and expose two different signatures:

```
Static list : *> *.
Static empty : (A : *) -> (list A).
Static new : (A : *) -> A> {(list A) :: |new |empty}> (list A).
List |A :: *> * => {(list A) :: |new |empty}.
NonEmpty |A :: *> * => {(list A) :: |new}.
```

Listing 1 Definition of empty and non-empty lists using first-class signatures

Rule 3 ensures that each constructor inhabits the declared signature, while Rule 4 provides $\mathsf{NonEmpty}\,A \sqsubseteq \mathsf{List}\,A$ because the latter merely adds the `empty` constructor. This formulation enables the definition of functions that operate on both general lists and non-empty lists:

```
length |A ls :: (A : *) -> (List A)> Nat =>
[ls of Nat
|(empty _) => 0
|(new A head tail) => (+1 (length A tail))
].
```

Listing 2 Length function compatible with both list variants

Alternatively, we can define functions that exclusively accept non-empty lists, directly mirroring the running example:

```
last |A ls :: (A : *) -> (NonEmpty A)> A =>
[ls of A
    |(new A head tail) => [tail of A
        |(empty _) => head
        |(new A head2 tail2) => (last A (new A head2 tail2))
    ]
].
insertsort |xs v :: (List Nat)> Nat> (NonEmpty Nat) =>
[xs of (NonEmpty Nat)
    |(empty _) => (new Nat v (empty Nat))
    |(new _ head tail) => [(gte head v) of (NonEmpty Nat)
        |false => (new Nat head (insertsort tail v))
        |true => (new Nat v (new Nat head tail))
    ]
].
def list_inserted_has_a_last_element |xs v :: (List Nat)> Nat> Nat =>
(last Nat (insertsort xs v))
```

Listing 3 Subtyping application to eliminate trivial proof obligations

Rules 2 and 7 explain why `last` needs only one clause: the type of `ls` is $\{(List\ A) :: |new\}$, so no `empty` branch is requested. The helper `list_inserted_has_a_last_element` demonstrates that the sorted list inherits the non-empty signature with no auxiliary proofs—exactly the two obligations eliminated relative to Figure 1.

One might assume the following representation is correct, as previously discussed, yet Rule 2 warns against using phantom predecessors:

```
Static nat : *.
Static 0 : nat.
Static +1 : nat> nat.
Nat :: {nat :: |0 |+1}.
```

Listing 4 Attempt with phantom predecessor

Here the predecessor of `+1` would live in $\{nat :: |0| + 1\}$ but the constructor type mentions only the phantom `nat`. The checker's algorithmic Rule 5 rejects this definition, forcing us to use the recursive variant below:

```
Static nat : *.
Static 0 : nat.
Static +1 : Nat> nat.
Nat :: {nat :: |0 |+1}.
```

Listing 5 Recursive definition accepted by the algorithmic Rule 5 check

Because the predecessor now mentions the signature `Nat`, Rule 3 can derive the constructor typing judgment while the AGAINST predicate keeps indices aligned. This also demonstrates how $\{T :: \Phi\}$ can refer to itself without reintroducing the proof obligations we set out to eliminate [10].

4 Implementation

Our implementation now has two components. The backend encoding in Lambdapi (`encode.lp`, 666 lines) defines signatures, constructor-list well-formedness, subtyping checks, and signature-directed elimination on top of the $\lambda\Pi$ calculus modulo rewriting [13]. The frontend transpiler (`Main.hs`, 956 lines) parses a restricted surface syntax and lowers it to backend terms. Both components are in `subindex/subtyping/`. The lowering mirrors Rules 1–7 from Section 2. Signature annotations become `signature_term`. Constructor introductions become `signature_intro`. Signature-only matches become `signature_case`. Appendix A shows representative input and generated output.

A Surface-to-Lambdapi Transpiler

The current artifact is organized around a translation pipeline: users write a compact surface fragment, and `siglp-transpiler` lowers it to backend terms consumed by `subtyping.encode`. This keeps examples short while preserving explicit generated code.

The supported surface fragment includes `Inductive` declarations, `Def` declarations, signature shorthand (`{T :: c_1 | ...| c_n}`), and signature-only `match` expressions. For each inductive declaration, the transpiler generates helper symbols for target, index shape, telescope, and constructor list. It rewrites term-level signatures to `signature_term`. It rewrites constructor applications to `signature_intro`. It rewrites signature matches to `signature_case`.

```
Static nat : *.
Static 0 : nat.
Static +1 : Nat -> nat.
Nat :: {nat :: |0 |+1}.

Static vec : * -> nat -> *.
Static empty : (s : *) -> (vec s 0).
Static cons : (s : *) -> (n : nat) -> (vec s n) -> (lift s) -> (vec s
     (+1 n)).

NonEmptyVecNat |v :: nat -> * => {(vec nat (+1 v)) :: |cons}.

head_nonempty |v m :: nat -> (NonEmptyVecNat v) -> Nat =>
[m of Nat
  |(cons k) => (+1 0)
].
```

Figure 4 Mini-language rendering of the non-empty vector example

Compiling with `cabal run siglp-transpiler` (input: `examples/sample.siglp`, output: `examples/sample.lp`) yields LambdaPi code such as Figure 5.

```
require open subtyping.encode;

symbol vec__sig_target : Set;
rule vec__sig_target ↪ vecLabel;
symbol vec__sig_indexes : indexes_type;
rule vec__sig_indexes ↪
  index_poly_succ (index_dependent_lift_succ natLabel (index_null));
symbol vec__sig_delta : delta_tel;
rule vec__sig_delta ↪
  mk_delta_tel vec__sig_indexes vec__sig_pattern_any;

symbol head_nonempty : Π (v : nat),
  (signature_term vec__sig_target
    (mk_delta_tel vec__sig_indexes (...))
    (constructor_list_cons consLabel (constructor_list_nil))) → nat;

rule head_nonempty ↪ λ (v : nat) (m : ...),
  (signature_case vec__sig_target (mk_delta_tel vec__sig_indexes (...))
    (constructor_list_cons consLabel (constructor_list_nil))
    natLabel m
    (case_branches_cons consLabel (constructor_list_nil) natLabel
      (λ (k : constructor_type consLabel), (+1 _0))
      (case_branches_nil natLabel)));
```

Figure 5 Generated LambdaPi excerpt after lowering signatures and matches

The complete generated files are omitted for space. Full examples are available in `subindex/subtyping/transpiler/examples/`, and the target encoding used by generated terms is `subindex/subtyping/encode.lp`.

Backend shallow encoding (`encode.lp`).

The backend is organized as a shallow encoding of Rules 1–7 into LambdaPi symbols and rewrite rules. Formation and well-formedness are captured by `signature_wfb` and `signature`. Subtyping uses boolean checkers (`signature_subtypeb`, `signature_to_base_subtypeb`) wrapped by `is_true` when a proposition is required. Elimination is represented directly by `signature_term`, `case_branches`, and `signature_case`. Finally, a coercion hook (`coerce_rule`) inserts signature widening automatically when constructor-list inclusion is validated.

B Conclusion

In this research, we introduced a compact yet powerful subtype system that demonstrates remarkable versatility and compatibility with various type theories. Our system effectively addresses the challenge of reusing and subtyping constructors in indexed datatypes through the novel introduction of first-class datatype signatures. A particularly notable feature of our approach is its ability to eliminate trivial proof obligations even when an arbitrary set of constructors is involved, significantly reducing the proof burden in practical applications. We have illustrated the system's flexibility through several practical examples drawn from real-world programming scenarios. Furthermore, we have provided a concise implementation that maintains simplicity without sacrificing expressiveness, making our approach accessible for integration into existing proof assistants and type systems.

C Future Work

There are several directions in which we would like to extend this work.

```
symbol signature_wfb : Set → delta_tel → constructor_list → Bool;
inductive signature : Set → delta_tel → constructor_list → TYPE ≔
| signature_nil : Π (T : Set) (g : delta_tel), signature T g constructor_list_nil
| signature_cons :
    Π (T : Set) (g : delta_tel) (c : labelConstructor) (cs : constructor_list),
    π (is_true (against_delta_b T g (constructor c))) →
    signature T g cs →
    signature T g (constructor_list_cons c cs);

symbol signature_subtypeb :
  Set → delta_tel → constructor_list →
  Set → delta_tel → constructor_list → Bool;

inductive signature_term : Set → delta_tel → constructor_list → TYPE ≔
| signature_intro :
    Π (T : Set) (g : delta_tel) (phi : constructor_list) (c : labelConstructor),
    π (is_true (constructor_in_signature_b T g phi c)) →
    constructor_type c →
    signature_term T g phi;

symbol signature_case :
  Π (T : Set) (g : delta_tel) (phi : constructor_list) (Q : Set),
  signature_term T g phi →
  case_branches phi Q →
  lift Q;

coerce_rule coerce_signature
  (signature $T $g $phi1)
  (signature $T $g $phi2)
  $x ↪ coerce_signature $T $g $phi1 $phi2 $x;
```

Figure 6 Shallow backend encoding excerpt from `encode.lp`

- *Mechanized meta-theory.* We plan to formalize the typing and subtyping rules, together with subject reduction and progress, in a proof assistant such as Coq or Agda, and connect the proofs to our prototype implementation.
- *Larger case studies.* Our current examples are small; applying constructor subset signatures to larger developments in existing proof assistants (e.g. libraries of lists and vectors) would help validate their practical benefits.
- *Richer signatures.* Finally, we intend to explore simple extensions of signatures with basic set operations on constructor sets (such as unions) while keeping the meta-theory and implementation lightweight.

References

1. D. Aspinall and A. Compagnoni. Subtyping dependent types. In *Proceedings 11th Annual IEEE Symposium on Logic in Computer Science*, pages 86–97, 1996. doi:10.1109/LICS.1996.561307.
2. Gilles Barthe. Order-sorted inductive types. *Information and Computation*, 149(1):42–76, 1999. URL: https://www.sciencedirect.com/science/article/pii/S0890540198927511, doi:10.1006/inco.1998.2751.
3. Gilles Barthe and Maria João Frade. Constructor subtyping. In *Proceedings of the 8th European Symposium on Programming Languages and Systems*, ESOP '99, page 109–127, Berlin, Heidelberg, 1999. Springer-Verlag.
4. Pierre-Évariste Dagand and Conor McBride. Elaborating inductive definitions. *CoRR*, abs/1210.6390, 2012. URL: https://arxiv.org/abs/1210.6390, arXiv:1210.6390.
5. Matthew Fluet and Riccardo Pucella. Practical datatype specializations with phantom types and recursion schemes, 2005. arXiv:cs/0510074.
6. Maria João Frade. Type-based termination of recursive definitions and constructor subtyping in typed lambda calculi. 2003. URL: https://api.semanticscholar.org/CorpusID:115763806.

7 Christopher Jenkins, Colin McDonald, and Aaron Stump. Elaborating inductive definitions and course-of-values induction in cedille, 2019. arXiv:1903.08233.

8 Andrew Marmaduke, Christopher Jenkins, and Aaron Stump. Zero-cost constructor subtyping. In Olaf Chitil, editor, *IFL 2020: 32nd Symposium on Implementation and Application of Functional Languages, Virtual Event / Canterbury, UK, September 2-4, 2020*, pages 93–103. ACM, 2020. doi:10.1145/3462172.3462194.

9 Rodrigo Marques, Mário Florido, and Pedro Vasconcelos. Towards algebraic subtyping for extensible records, 2024. URL: https://arxiv.org/abs/2407.06747, arXiv:2407.06747.

10 Per Martin-Löf. Intuitionistic type theory by per martin-löf. notes by giovanni sambin of a series of lectures given in padova, june 1980. 2021.

11 Johan Nordlander. Polymorphic subtyping in o'haskell. *Science of Computer Programming*, 43(2-3):93–127, 2002. Validerad; 2002; 20070227 (ysko). doi:10.1016/S0167-6423(02)00026-6.

12 Ulf Norell. Towards a practical programming language based on dependent type theory. 2007. URL: https://api.semanticscholar.org/CorpusID:118357515.

13 Ronan Saillard. Typechecking in the lambda-pi-calculus modulo : Theory and practice. (vérification de typage pour le lambda-pi-calcul modulo : théorie et pratique). 2015. URL: https://api.semanticscholar.org/CorpusID:2853829.